

Corso di Programmazione Web

Simulazione di esame – 4 Aprile 2025

- Questo esame è a libro aperto, ma tutto il lavoro deve essere svolto individualmente.
- Rispondere a ogni domanda in modo chiaro, utilizzando schizzi, tabelle o descrizioni dove necessario.
- Fornire esempi di richieste e risposte per mostrare il funzionamento.
- Non scrivere codice effettivo, ma fornire dettagliate progettazioni API, i formati di richiesta/risposta e i modelli di dati.

Ci è stato richiesto di definire un servizio di biblioteca in cui possiamo ottenere la lista dei libri e mettere delle recensioni. Il servizio si compone delle seguenti API:

- Una API per ottenere la lista di libri;
- Una API per ottenere il dettaglio di un singolo libro;
- Una API per inserire una recensione per il libro scelto.

Si osservi che la risorsa “libro” può essere riutilizzata in diverse di queste API. Si suggerisce di assegnare un identificativo univoco a ogni libro, e di aggiungere almeno due campi, oltre alla recensione. Si assuma che ogni libro possa avere solo una recensione.

Domanda 1: Definizione delle API (15 punti)

- Definire gli endpoint API (strutture URL e metodi HTTP); **(2.5 punti per ogni API)**
- Specificare i parametri di richiesta e le possibili risposte, possibilmente con un esempio di richiesta andata a buon fine; **(1.5 punti per ogni API)**
- Dettagliare i modelli di dati previsti per le richieste e le risposte. **(3 punti in totale)**

Domanda 2: Gestione degli Errori API (5 punti)

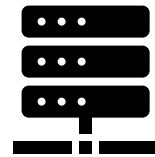
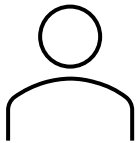
Una buona API deve gestire efficacemente gli errori.

- Descrivere le risposte di errore comuni (es. 400, 404, 500) e quando dovrebbero essere utilizzate nel contesto delle API appena progettate. **(3 punti)**
- Fornire esempi di risposta per almeno due casi di errore. **(1 punto per ogni esempio)**

Corso di Programmazione Web
Simulazione di esame – 4 Aprile 2025
Possibile soluzione

Si presenta di seguito una proposta di soluzione. Si osservi che questa presenta solo uno spunto di progettazione, ma non va intesa come l'unica soluzione possibile.

Progettazione API di richiesta lista di libri



GET /books →
← 200, [{Book}]

Si prevede di implementare una richiesta GET che restituisce una lista di libri. La struttura dati inviata dal server come risposta sarà una lista, i cui elementi sono oggetti di tipo libro. Si specificano i campi dell'oggetto libro:

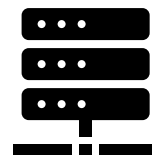
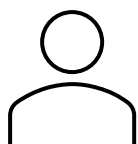
Book

- id: int
- title: string
- author: string
- review: int
 - minimum: 1
 - maximum: 5

Un esempio di risposta può essere il seguente:

```
[  
  {  
    "id": 1,  
    "title": "Don Quixote",  
    "author": "Miguel de Cervantes",  
    "review": 5  
  }  
]
```

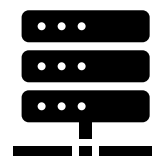
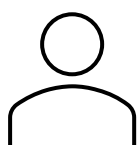
Progettazione API per ottenere il dettaglio di un singolo libro



GET /books/{id} →
← 200, {Book}

L'API per richiedere il singolo libro può essere costruita in modo gerarchico a partire dalla API precedente. In questo caso, il libro viene individuato dal query parameter "id", che individua la singola risorsa. Anche per questa API, che richiede informazioni e non richiede l'invio di dati da parte dell'utente, si sceglie l'utilizzo della richiesta GET. In caso di successo, l'API restituisce un singolo oggetto di tipo Book, analogo a quello specificato sopra.

Progettazione API per l'inserimento di una recensione



POST /books/{id}/recensione {"recensione": 5} →
← 200

L'API di inserimento di una recensione può accedere alla risorsa libro tramite lo stesso path usato nella precedente API, e in modo gerarchico accedere alla sotto-risorsa "recensione". Inoltre, per l'invio di una recensione, si sceglie di incorporare il dato inviato dall'utente in una richiesta POST, in quanto comporta l'inserimento di una nuova informazione nel server. Si sceglie di limitare l'utente nella scelta dei valori possibili per la recensione, richiedendo un valore tra 1 e 5.

Gestione degli errori

Le risposte di errore sono usate dal server per comunicare all'utente diversi tipi di errori riscontrati durante l'esecuzione delle richieste.

- *Errore 400*. Questo errore caratterizza una richiesta errata da parte dell'utente. Per esempio, nel caso delle API appena progettate, potrebbe essere restituito qualora l'utente invii un valore maggiore di 5 nel body della richiesta POST per l'inserimento della recensione, o qualora invii una richiesta con il body vuoto.

- *Errore 404.* Questo errore è utilizzato per comunicare all'utente che non solo la richiesta è errata, ma che la risorsa richiesta non esiste. Può essere restituito da API a endpoint non esistenti (per esempio `/utenti`), o anche nel caso l'utente specifichi un parametro `{id}` non esistente nella lista di libri.
- *Errore 500.* Questo errore viene usato per indicare un errore lato server. Per esempio, può essere restituito dall'API per l'inserimento della recensione se il server non è in grado di salvare l'informazione inviata dall'utente. Un altro esempio potrebbe essere qualora la lista di libri diventasse troppo grande da gestire e facesse bloccare il server inavvertitamente.